

Debugging, Profiling, Tracing et Analyse de performance sous Linux

Analyse mémoire

Objectifs du module

À la fin de ce module, vous serez capable de :

- Lire correctement RSS, VSZ, shared, cache et buffers
- Exploiter /proc pour auditer la consommation d'un processus
- Identifier une fuite mémoire d'une croissance légitime
- Anticiper un déclenchement de l'OOM Killer

Plan

1. Modèle mémoire Linux et /proc
2. Outils free, vmstat, pmap
3. Fuites et OOM Killer
4. Démo et TP
5. Récap et questions

1. Modèle mémoire Linux

Les chiffres qui trompent

“ « free montre 500 Mo libre, pourtant j'ai 8 Go. » C'est normal. ”

- Le noyau utilise la RAM disponible pour le cache page
- `available` est la vraie valeur à regarder, pas `free`
- `RSS` $\neq$ empreinte exclusive, il inclut la mémoire partagée
- `VSZ` peut être énorme (mmap) sans conso physique

Les champs clés de /proc//status

```
VmSize:      1523456 kB    (VSZ, espace adressable)
VmRSS:       245678 kB    (résidente en RAM)
VmData:      123456 kB    (data + heap)
VmStk:        132 kB     (pile)
VmExe:        512 kB     (code exécutable)
VmLib:        8192 kB     (libs)
RssAnon:     210000 kB    (résidente anonyme, typique fuite)
RssFile:      35678 kB    (résidente mappée fichier)
```

RssAnon qui grimpe sans borne = candidat fuite.

2. Outils free, vmstat, pmap

free bien lu

```
free -h
```

	total	used	free	shared	buff/cache	available
Mem:	15Gi	3.2Gi	420Mi	120Mi	11Gi	11Gi
Swap:	2.0Gi	0B	2.0Gi			

- `used` inclut la mémoire noyau
- `buff/cache` est récupérable par le noyau
- **La colonne qui compte : `available`**

vmstat pour la pression mémoire

```
vmstat 1 10
```

- `si` / `so` non nuls = swap actif, la machine souffre
- `free` qui descend pendant que `cache` monte = comportement sain
- `free` qui descend ET `cache` stable = process qui consomme
- `b` (blocked) élevé sur mémoire = attente d'I/O pour pages

pmap pour voir les mappings

```
pmap -x <pid>
```

- Liste toutes les zones mmap avec leur taille et permissions
- Repérer les grosses zones anonymes (heap, mmap)
- Comparer deux snapshots pour isoler la zone qui grossit

3. Fuites et OOM Killer

Détecter une fuite

Démarche systématique :

1. Capturer `VmRSS` et `RssAnon` toutes les N secondes
2. Vérifier la croissance monotone sur une charge constante
3. Faire un diff `pmap` entre T0 et T0+30min
4. Pour confirmer : `valgrind --tool=memcheck` hors prod, ou `heaptrack`

OOM Killer

Quand la RAM + swap sont saturées :

- Le noyau choisit un processus à tuer (score dans `/proc/<pid>/oom_score`)
- Message visible dans `dmesg` : `Out of memory: Killed process ...`
- Ajuster via `oom_score_adj` pour protéger un process critique

```
dmesg -T | grep -i 'killed process'  
cat /proc/<pid>/oom_score  
echo -500 > /proc/<pid>/oom_score_adj
```

TP à suivre

TP 2 : analyser une consommation mémoire anormale

- Durée : 1h
- Fichier : `tp/tp-02.md`

